REFERENCE VOLUME

Kubernetes Security

for production workloads

Deep-technical controls across the cluster lifecycle, mapped to Amazon EKS, Google GKE, and Azure AKS.

Coverage

- Threat model & shared responsibility across managed K8s
- Control-plane: API server, etcd, audit, admission pipeline
- AuthN/AuthZ: OIDC, IRSA / Workload Identity / Workload ID
- Pod Security Standards, OPA Gatekeeper, Kyverno, VAP
- Network: CNI, NetworkPolicy, service mesh, egress
- Supply chain: signing, SBOM, admission verification
- Secrets: external secret stores, KMS, encryption at rest
- Runtime: eBPF, Falco, native runtime defense
- Multi-tenancy patterns from namespace to dedicated cluster
- Full EKS / GKE / AKS feature matrix

Companion to: AWS / Azure / Google Cloud reference volumes · IAM for Multi-Cloud SaaS · Cross-Cloud Service Mapping

Principal Security Architect · Technical Leadership Edition

Contents

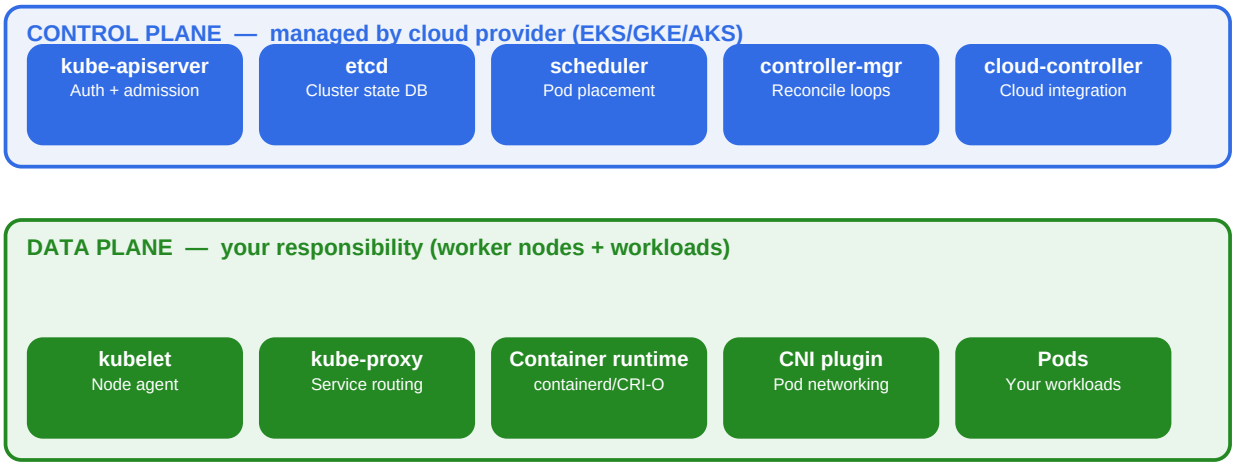
How to read this reference

Each section explains a security domain at depth, then maps the controls to Amazon EKS, Google GKE, and Azure AKS. Code snippets are real (YAML, Rego, kubectl) and tested-shape representative. Where vendor naming differs, the EKS / GKE / AKS comparison in section 12 collapses the three into a single decision matrix.

1. The Kubernetes threat model

Where attackers strike, and why a defense-in-depth posture is non-negotiable for production clusters.

Kubernetes attack surface — components and where attackers target



Common attack vectors:

1 Exposed/unauth API server → cluster takeover
2 Stolen kubeconfig / over-privileged service account
3 Container escape via privileged pod or kernel exploit
4 Supply chain: malicious image, dependency, or admission webhook

Why Kubernetes is a high-value target

A Kubernetes cluster concentrates extraordinary privilege in one place: it holds the deployment specifications, secrets, and runtime credentials for many applications, and its API server is a single control point that can schedule arbitrary code on any node. A successful attack on the cluster typically yields access to the workloads, the data those workloads can reach, and lateral movement across every other component in the cluster's trust boundary. Most cloud breaches involving Kubernetes follow one of the four canonical paths shown in the diagram above.

The four canonical attack paths

Path	How it starts	How it escalates	Primary mitigation
Exposed API server	API server on public internet with weak/missing authn	Direct cluster takeover — deploy malicious pods, exfiltrate secrets	Private API endpoint; authorized networks; mTLS; AWS IAM/IRSA bound
Stolen kubeconfig / service-account token	Token leaked from CI logs, developer laptop, public repo	Token enables API actions; over-privileged tokens enable cluster takeover	Short-lived tokens (BoundServiceAccountTokenVolume); IRSA/WI; OIDC SSO
Container escape	Privileged pod, missing seccomp, kernel CVE, hostPath mount	Break out to node; access kubelet credentials; pivot to control plane	PSA restricted; no privileged pods; seccomp/AppArmor; gVisor for hostile workloads

Path	How it starts	How it escalates	Primary mitigation
Supply chain compromise	Malicious base image, dependency, admission webhook, or operator	Code runs at deploy or runtime with cluster privileges	Image signing (Sigstore); Binary Authorization; SBOM; scan everything

MITRE ATT&CK; for Containers — mapping the kill chain

MITRE ATT&CK; for Containers provides a structured taxonomy of attacker behaviors observed in Kubernetes environments. The most relevant techniques for production cluster defense map cleanly onto the controls in this reference:

Tactic	Example technique	Defended by
Initial Access	Exploit Public-Facing Application; External Remote Services	Network policies, ingress hardening, WAF in front of services
Execution	Container Administration Command; Deploy Container	RBAC, admission control, image signing verification
Persistence	Implant Internal Image; Scheduled Task/Job	Image admission, RBAC on Jobs/CronJobs, runtime detection
Privilege Escalation	Escape to Host; Privileged Container	PSA restricted, seccomp, AppArmor, runtime detection (Falco)
Defense Evasion	Build Image on Host; Disable or Modify Tools	Read-only root FS, audit logging, runtime integrity monitoring
Credential Access	Steal Application Access Token; Container API	IRSA/WI for short-lived credentials, secrets in external stores
Discovery	Container and Resource Discovery	RBAC denies cross-namespace listing; network policies block recon
Lateral Movement	Use of cloud credentials; Internal scanning	NetworkPolicy default-deny, mTLS via mesh, segregated node pools
Impact	Resource Hijacking (crypto-mining); Data Destruction	Resource quotas, runtime detection, immutable backups

2. Shared responsibility on managed Kubernetes

Every managed Kubernetes service hands you a partial cluster. Knowing exactly where the line falls is the foundation of a working security program.

Shared responsibility — managed Kubernetes (EKS / GKE / AKS)

CLOUD PROVIDER manages

- Control plane availability & patching
- etcd encryption at rest (managed key)
- API server endpoint TLS
- Control plane network isolation
- Underlying hardware & hypervisor
- Auto-mode node OS patching (GKE Autopilot / EKS Auto Mode / AKS auto-upgrade)

YOU manage

- Worker node OS hardening & patching (unless auto-mode)
- RBAC, namespaces, ServiceAccounts
- Pod Security & admission policies
- Network policies, mesh, egress controls
- Workload images & supply chain
- Secrets management & encryption keys
- Logging, audit retention & detection
- Application-level authn/authz

Where the line falls — in detail

EKS, GKE, and AKS all manage the control plane (API server, etcd, scheduler, controller manager) but differ in how much of the data plane they handle for you. The newer “auto” modes — GKE Autopilot, EKS Auto Mode, AKS Automatic — move the node-management responsibility back to the provider in exchange for less flexibility. Knowing which mode you run is the first security question, because it changes who patches your nodes.

Responsibility	EKS	GKE	AKS
Control plane patching	AWS — you trigger minor-version upgrades	Google — automatic or release-channel-driven	Microsoft — you trigger minor-version upgrades
etcd encryption at rest (managed key)	Default on; AWS-managed KMS CMK	Default on; Google-managed key	Default on; platform-managed key
etcd encryption with customer-managed key	Optional via KMS CMK (envelope encryption for secrets)	Optional via Cloud KMS (application-layer secrets encryption)	Optional via Azure Key Vault (KMS plugin / disk-level)
Worker node OS patching	You (managed node groups assist); EKS Auto Mode — AWS	You (with auto-upgrade); Autopilot — Google	You (with auto-upgrade); AKS Automatic — Microsoft

Responsibility	EKS	GKE	AKS
Worker node hardening (Bottlerocket / COS / Mariner)	Bottlerocket (recommended) or AL2/AL2023	Container-Optimized OS (default)	Azure Linux (Mariner, recommended) or Ubuntu
RBAC, NetworkPolicy, admission policy	You	You (some default policies in Autopilot)	You
Workload supply chain (images, signing)	You	You (Binary Authorization is the integrated tool)	You
Audit log generation	Provider generates; you enable and route	Provider generates; you enable and route	Provider generates; you enable and route
Audit log retention & analysis	You (CloudWatch / S3 / SIEM)	You (Cloud Logging / BigQuery / SIEM)	You (Log Analytics / Sentinel / SIEM)

3. Control plane security

The API server and etcd are the cluster's crown jewels. Everything that happens in the cluster passes through them.

API server access control

The kube-apiserver is the only component that talks to etcd, and every other component (kubelet, scheduler, controllers, kubectl, operators) reaches the cluster through it. Two questions define API server security: **who can reach it on the network**, and **who can authenticate to it**.

Private API endpoints — the single highest-leverage control

Publicly exposed API servers remain one of the most common Kubernetes breach causes — a misconfigured cluster on the open internet, with weak or default authentication, is exploited within hours. Every production cluster should have its API server on a private endpoint, with the public endpoint either disabled entirely or restricted to a tightly scoped allow-list.

Control	EKS	GKE	AKS
Private endpoint	Private cluster: API in VPC only	Private cluster: master in Google-managed VPC, peered to yours	Private cluster: API in your VNet via Private Endpoint
Public endpoint allow-list	Public access CIDRs (publicAccessCidrs)	Authorized networks (master-authorized-networks)	API server authorized IP ranges
Egress requirement for nodes	Nodes need outbound to API endpoint, ECR, S3	Nodes use private Google access; Cloud NAT for external	Nodes need outbound to API, ACR, Azure services
Disable public endpoint entirely	Yes — EndpointPublicAccess=false	Yes — --enable-private-endpoint	Yes — set apiServerAccessProfile to private only

etcd encryption

etcd holds the cluster's entire state, including Secrets. Cloud providers encrypt etcd at rest with a managed key by default — fine for low-sensitivity workloads, but for regulated or multi-tenant clusters you should layer envelope encryption with a customer-managed key. This means Secrets are encrypted with a data key, which is itself encrypted with your KMS CMK before being written to etcd. Even if an attacker exfiltrates the etcd database, the Secrets remain ciphertext without access to your CMK.

```
# EKS - encrypt Secrets with a customer-managed KMS key
aws eks create-cluster \
  --name prod \
  --encryption-config '[{"provider":{"keyArn":"arn:aws:kms:..."},"resources":["secrets"]}]'

# GKE - application-layer secrets encryption via Cloud KMS
gcloud container clusters create prod \
  --database-encryption-key projects/$P/locations/$L/keyRings/$KR/cryptoKeys/$K

# AKS - KMS plugin with Azure Key Vault
az aks update -n prod -g rg --enable-azure-keyvault-kms \
  --azure-keyvault-kms-key-id https://$VAULT.vault.azure.net/keys/$KEY
```

Control plane audit logging

Every API call to the cluster generates an audit event. These events are **the single most valuable source of forensic evidence** when an incident occurs — they show exactly which identity made which call, when, and what the response was. All three managed services emit audit events to the cloud's native logging platform; you must enable, route, and retain them.

Audit feature	EKS	GKE	AKS
Native destination	CloudWatch Logs (per log type)	Cloud Logging (admin / data access / system)	Azure Monitor / Log Analytics workspace
Log types enabled separately	api, audit, authenticator, controllerManager, scheduler	ADMIN_READ, DATA_READ, DATA_WRITE per service	kube-audit, kube-audit-admin, kube-apiserver, etc.
Retention default	CloudWatch retention (configurable)	Cloud Logging default 30/400 days by tier	Log Analytics workspace retention
Recommended retention	≥ 1 year for audit; archive to S3 with Object Lock	≥ 1 year for audit; sink to GCS/BigQuery with retention	≥ 1 year for audit; archive to immutable storage

4. Authentication and authorization

Three identity systems intersect in every cluster: the cloud's IAM, Kubernetes' native identities, and the workload's runtime identity. Getting them aligned is most of the security work.

The three identity layers

Layer	Identifies	Authenticates via	Authorizes via
Cloud IAM	Human and machine identities in the cloud account	Cloud-native (SSO, access keys, OIDC federation)	Cloud IAM policies
Kubernetes RBAC	Users, groups, ServiceAccounts inside the cluster	Bearer tokens, certificates, OIDC tokens	Roles, ClusterRoles, RoleBindings, ClusterRoleBindings
Workload identity (runtime)	The pod / workload itself when calling cloud APIs	Pod's ServiceAccount token → cloud OIDC exchange	Cloud IAM role attached to the workload identity

Human access to clusters — stop using long-lived kubeconfigs

The classic workflow — a developer with a long-lived kubeconfig file on their laptop containing static credentials — is one of the most common sources of cluster breaches. The credentials leak through repos, CI logs, lost laptops, or insider activity, and they typically have far more permissions than the developer ever uses. The modern pattern is **SSO-driven, short-lived token-based access** backed by the cloud IAM.

Pattern	EKS	GKE	AKS
Native SSO integration	aws-auth ConfigMap → IAM roles map to K8s groups; or EKS Access Entries (newer)	Google groups map to K8s RBAC via Google Groups for RBAC	Azure AD (Entra ID) integration — native
Short-lived token mechanism	aws eks get-token — STS-backed; refreshed on use	gcloud config helper — OAuth refresh; short-lived	kubelogin / az aks get-credentials — Entra ID tokens
Group-driven RBAC	IAM role/Identity Center group → K8s group binding	Google group → K8s RBAC binding	Entra ID group → K8s RBAC binding
Just-in-time access (preferred)	IAM Identity Center session policies + time-bound role assumption	PAM-style: Cloud IAM conditional access; HashiCorp Boundary/Teleport	Entra Privileged Identity Management (PIM) for elevated access

Workload identity — the canonical pattern for cloud API access

Workloads in your cluster routinely need to call cloud APIs: read from object storage, decrypt with KMS, write to a database, fetch a secret. The wrong way is to bake a long-lived access key into the container image or a Kubernetes Secret. The right way is **OIDC-based workload identity**, which gives each pod a short-lived, IAM-bound, cryptographically-verified identity through the pod's ServiceAccount.

How it works (the unified model)

- The cluster's API server is configured as an **OIDC identity provider**. Its JWKS endpoint publishes the public keys that sign ServiceAccount tokens.
- The cloud's IAM is configured to **trust that OIDC provider**. An IAM role/identity declares a trust policy that accepts tokens from this specific cluster issuer and a specific ServiceAccount.
- A pod mounts a **projected ServiceAccount token** (audience-bound, time-bound, automatically rotated by the kubelet).
- The pod's SDK exchanges the projected token for cloud credentials via the cloud's STS / token endpoint, getting **short-lived (typically <1 hour) IAM credentials** scoped to that role.
- All access is logged in the cloud's audit trail with the cluster + ServiceAccount as the principal — you can trace every cloud API call back to a specific pod.

Workload identity mechanism	EKS	GKE	AKS
Modern name	EKS Pod Identity (newer) / IRSA (older)	GKE Workload Identity	Azure Workload Identity
Trust mechanism	OIDC provider per cluster; trust policy on IAM role	OIDC — cluster trusts Google IAM; SA → GSA binding	OIDC — federated credentials on Azure AD app/managed identity
Credential lifetime	Default 1h, configurable	Default 1h, configurable	Default 1h, configurable
Annotation on K8s SA	eks.amazonaws.com/role-arn (IRSA) or pod-identity association	iam.gke.io/gcp-service-account	azure.workload.identity/client-id (label, not annotation)
Replaces	Long-lived AWS access keys in pods	Long-lived GCP service account JSON keys	Long-lived Azure secrets / managed-identity issues

RBAC — the core authorization model

Kubernetes RBAC controls what authenticated identities can do inside the cluster. The model is straightforward but the discipline is everything: most clusters drift toward over-permissive RBAC because creating a narrow Role is harder than reusing `cluster-admin`. The result is that compromise of any account often equals compromise of the cluster.

The four RBAC primitives

- **Role** — namespace-scoped set of permissions (verbs on resources).
- **ClusterRole** — cluster-wide set of permissions; can also be used in namespaces.
- **RoleBinding** — grants a Role to a subject (User, Group, or ServiceAccount) in a namespace.
- **ClusterRoleBinding** — grants a ClusterRole to a subject across the entire cluster.

RBAC anti-patterns to never commit

- **Binding cluster-admin to anyone except a tiny number of break-glass identities.** Anyone with cluster-admin can read every Secret, deploy any pod, and impersonate any ServiceAccount.
- **Wildcard verbs or wildcard resources** in production Roles. `verbs: ["*"]` is almost always wrong.
- **Granting permissions on Secrets at the namespace level when only a subset is needed.** If a workload only needs one Secret, name it explicitly with `resourceNames`.
- **The default ServiceAccount with mounted credentials.** Every pod gets the namespace's `default` ServiceAccount unless you specify otherwise. Disable token mounting for pods that don't need it (`automountServiceAccountToken: false`).
- **Permissions to `create` on `pods/exec` or `impersonate`** on non-debugging identities. These are privilege-escalation primitives.

RBAC review and detection

RBAC sprawl is detectable. Tools and queries should be part of your monthly hygiene:

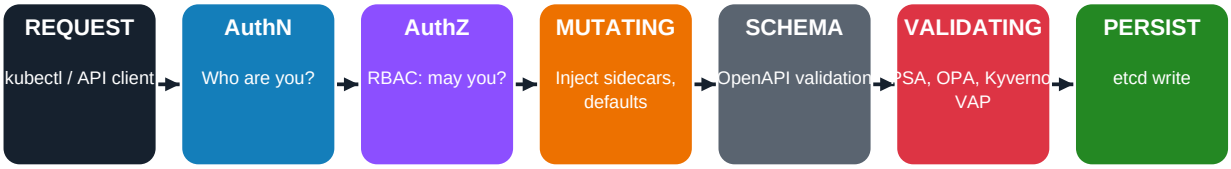
- **kubectl who-can** (krew plugin) — "who can `get secrets` in this namespace?"
- **rbac-tool** / **rakkess** — visualize and audit RBAC across the cluster.
- **kube-bench** — CIS Kubernetes Benchmark compliance check.
- **Continuous validation** — admission policies that reject overly broad bindings (e.g. block any binding to `cluster-admin` outside an allowlist).

> **The single most important RBAC review**
>
> Periodically list every binding to **cluster-admin** and every ClusterRoleBinding to **system:masters**. These are the keys to the kingdom. Anyone or anything in those lists can compromise the cluster trivially. The list should be tiny, fully justified, and reviewed quarterly.

5. Pod Security and admission control

The admission pipeline is where most cluster-level security policy is actually enforced — every API request flows through it, and that is where you accept or reject what runs.

Admission control — every API request flows through these stages



Where you enforce policy:

- Mutating webhooks — inject sidecars (service mesh, secret CSI), set defaults, label namespaces.
- Validating webhooks (OPA Gatekeeper, Kyverno, ValidatingAdmissionPolicy) — reject non-compliant resources.
- Pod Security Admission (PSA) — built-in label-driven enforcement of Pod Security Standards (privileged/baseline/restricted).
- Image admission (e.g. Sigstore, Kyverno verifyImages) — cryptographically verify signatures before allowing pods to run.

Pod Security Standards (PSS) — the new baseline

PodSecurityPolicy was removed in Kubernetes 1.25. Its replacement is **Pod Security Admission (PSA)**, a built-in admission controller that enforces three pre-defined profiles — *privileged*, *baseline*, and *restricted* — on a per-namespace basis via labels. PSA is simpler than PSP, but it's also less flexible. For policies beyond the three built-in profiles, use OPA Gatekeeper, Kyverno, or ValidatingAdmissionPolicy.

Profile	What it allows	When to use
privileged	Anything. No restrictions.	Cluster-level workloads only (CNI agents, kube-system). Never your apps.
baseline	Blocks known privilege escalations (privileged containers, hostNetwork, hostPath in most forms, capabilities additions beyond a minimal set)	Default for any namespace that isn't running fully-trusted apps
restricted	Heavily hardened: non-root, read-only root FS recommended, drop ALL capabilities, seccomp RuntimeDefault, no host namespaces	Default for all application namespaces in production

Enforcing PSA — the right way

```
# Enforce 'restricted' on the app namespace, warn on 'baseline' violations
apiVersion: v1
kind: Namespace
metadata:
  name: prod-app
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

Cluster-wide enforcement

Setting PSA labels on namespaces individually is error-prone. Use a cluster-wide default (in EKS / GKE / AKS, configure the PSA admission controller's defaults), plus a Validating policy that **requires every namespace to carry a pod-security enforce label**. New namespaces created without a label fail admission, closing the most common gap.

Beyond PSA — policy engines

PSA covers the basics. For real production policy — image registry allow-lists, label requirements, resource limits, label-based tenant isolation, GitOps integration, custom rules — you need a real policy engine. The three serious options are OPA Gatekeeper, Kyverno, and the new built-in ValidatingAdmissionPolicy.

Engine	Language	Strengths	When to choose
OPA Gatekeeper	Rego	Mature, multi-cluster CNCF graduate, policy library, audit mode	Org standardized on OPA across stack; complex policies; cross-engine consistency
Kyverno	YAML (native K8s style)	No new language; mutate + validate + generate; image verification built-in	Want K8s-native ergonomics; image signing/SBOM verification key requirement
ValidatingAdmissionPolicy (VAP)	CEL (Common Expression Language)	Built into K8s 1.30+; no webhook latency; in-tree, no extra deployment	Newer clusters; want to reduce webhook dependency; simple validation rules

Example: enforcing a signed-image-only policy

A canonical production policy: only allow images from your trusted registry, signed by your build pipeline. Here's how the same intent looks in each engine:

Kyverno (verify signature with Sigstore/cosign)

```
apiVersion: kyverno.io/v2beta1
kind: ClusterPolicy
metadata:
  name: require-signed-images
spec:
  validationFailureAction: Enforce
  rules:
  - name: verify-cosign
    match:
      any:
      - resources:
          kinds: [Pod]
    verifyImages:
    - imageReferences:
      - "registry.yoursaas.com/*"
      attestors:
      - entries:
        - keyless:
            subject: "https://github.com/yourorg/.*"
            issuer: "https://token.actions.githubusercontent.com"
```

OPA Gatekeeper (Rego — registry allow-list)

```
package k8strustedimages

violation[{"msg": msg}] {
  input.review.kind.kind == "Pod"
  image := input.review.object.spec.containers[_].image
  not startswith(image, "registry.yoursaas.com/")
  msg := sprintf("image %v is not from the trusted registry", [image])
}
```

ValidatingAdmissionPolicy (CEL — native, no webhook)

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: trusted-image-registry
spec:
  matchConstraints:
    resourceRules:
    - apiGroups: [""]
      apiVersions: ["v1"]
      operations: ["CREATE","UPDATE"]
      resources: ["pods"]
  validations:
  - expression: "object.spec.containers.all(c, c.image.startsWith('registry.yoursaas.com/'))"
    message: "Containers must use the trusted registry"
```

Admission policy in production — operational discipline

- **Start in audit mode**, not enforce. New policies almost always have edge cases; an aggressive enforce-mode rollout breaks legitimate workloads. Audit → review failures → enforce.
- **Namespace-scoped exceptions**, not policy disabling. Exemptions should be explicit and documented (e.g. kube-system exempt from PSA restricted because CNI agents legitimately need host access).
- **Test policies in CI**. OPA and Kyverno both support unit testing; run policies against representative manifests in CI before they reach a cluster.
- **Monitor admission denials.** A spike in denials may indicate a new threat (an attacker testing pods) or a broken workload (your own developers being blocked). Both deserve attention.
- **Plan for the webhook outage.** If your admission webhook is down and policy is `failurePolicy: Fail`, the cluster stops accepting changes. Design HA: multiple replicas, anti-affinity, fast-failing health checks, and a documented break-glass procedure.

6. Network security

Kubernetes is flat by default. Every pod can reach every other pod, every Service, and the outside world unless you say otherwise. Default-deny is not the default; you have to build it.

The CNI is your network

The Container Network Interface (CNI) plugin handles pod networking — IP assignment, routing, and (sometimes) policy enforcement. The choice of CNI determines what network security features are even possible. On managed Kubernetes you typically choose between the cloud's native CNI (AWS VPC CNI, GKE's native VPC routing, Azure CNI) and a feature-rich alternative like Cilium or Calico that supports advanced policy.

CNI capability	EKS	GKE	AKS
Default CNI	AWS VPC CNI (pod IPs from VPC)	GKE Dataplane V2 (Cilium-based) on new clusters; routes-based on older	Azure CNI (pod IPs from VNet) or kubenet
Cilium / eBPF dataplane	Optional add-on (Cilium); or native AWS VPC CNI with Network Policy	Default for GKE Dataplane V2 — Cilium under the hood	Azure CNI Powered by Cilium (newer) or BYO Cilium
Calico	Supported via add-on	Supported	Supported via Azure CNI overlay + Calico
NetworkPolicy enforcement	Requires Network Policy add-on or Calico/Cilium	Native (Dataplane V2)	Native (Azure NPM or Cilium)
Pod IPs routable from VPC	Yes (VPC CNI native)	Yes (alias IPs)	Yes (Azure CNI) or No (kubenet)

NetworkPolicy — the K8s default-deny lever

Kubernetes NetworkPolicy is the native object for restricting pod traffic. It is **opt-in**: if no NetworkPolicy selects a pod, all traffic is allowed. Once any policy selects a pod, only traffic matching a policy is allowed — default-deny is therefore implemented by applying a policy that matches all pods but allows nothing, then layering specific allow policies on top.

```
# Default deny all ingress + egress for the entire namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: prod-app
spec:
  podSelector: {}      # selects every pod in the namespace
  policyTypes: [Ingress, Egress]
  # no ingress or egress rules → nothing allowed

---
# Allow only the frontend to talk to the API on port 8080
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow-frontend
  namespace: prod-app
spec:
  podSelector:
    matchLabels: {app: api}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels: {app: frontend}
    ports:
    - protocol: TCP
      port: 8080
```

Limitations of plain NetworkPolicy

- **L4 only** — you can allow/deny by IP, port, and selector, but you can't say “this pod can call GET /api/v1/* but not POST”.
- **No identity-based policy beyond labels** — traffic identity is implicit in the pod's labels and network position, not cryptographic.
- **No egress to external services by DNS name** — standard NetworkPolicy doesn't resolve DNS. For “allow egress to api.stripe.com” you need Cilium FQDN policies or a service mesh.

Cilium and CiliumNetworkPolicy — the modern upgrade

Cilium (now the default in GKE Dataplane V2 and an option on EKS/AKS) is an eBPF-based CNI that extends Kubernetes networking with L7-aware policy, identity-based security, and rich observability via Hubble. CiliumNetworkPolicy is a superset of standard NetworkPolicy and supports:

- **L7 policy** — HTTP method/path, gRPC, Kafka topic, DNS queries.
- **FQDN egress policy** — allow egress to `api.stripe.com` without managing IP ranges.
- **Identity-based policy** — every endpoint has a cryptographic identity, used by mTLS and policy.
- **Cluster-wide and tenant-wide policy** — ClusterwideNetworkPolicy applies to all namespaces.
- **Visibility** — Hubble provides per-flow observability including dropped packets and the policy verdict.

Service mesh — when to add one

A service mesh (Istio, Linkerd, Cilium Service Mesh) adds three security capabilities that are hard to replicate otherwise: **mTLS between all pods automatically**, **fine-grained L7 authorization**, and **traffic-level observability**. The cost is operational complexity — every pod gets a sidecar (or ambient/sidecarless equivalent), the control plane needs HA, and certificate management runs continuously.

When you need a service mesh	Why
mTLS everywhere as a hard requirement (compliance, zero-trust)	The mesh auto-rotates certificates and enforces mTLS without app code changes
Fine-grained service-to-service authorization	AuthorizationPolicy: which service can call which endpoint with which method
Multi-cluster mesh / federated identity across clusters	Shared trust domain via SPIFFE; cross-cluster mTLS
Rich observability without instrumenting every service	Sidecar captures traffic and emits metrics, traces, access logs

> **The honest trade-off**
>
> Service meshes deliver real security and observability gains, but operating one well is a non-trivial commitment — control-plane upgrades, sidecar memory overhead, debugging an extra hop, and learning curve. Don't add a mesh unless you have a use case it solves better than the alternatives (NetworkPolicy + Cilium L7 + workload identity often gets you 70% of the way for a fraction of the operational cost). When you do adopt one, treat it as a platform commitment, not a feature flag.

Egress control — the often-forgotten direction

Most teams focus NetworkPolicy on ingress (what can call my pods), but egress (what my pods can call) is at least as important for stopping data exfiltration and command-and-control callbacks. A compromised pod with unrestricted egress can reach any internet endpoint.

- **NetworkPolicy egress rules** — explicit allow-lists for what each workload can talk to. Default-deny egress by namespace.
- **Cilium FQDN policy** — allow egress by DNS name (“can call api.stripe.com”) without tracking IP ranges.
- **Egress gateway / NAT** — force all cluster egress through a controlled point where you can inspect, log, and apply WAF-like rules.

- **DNS firewall** — block resolution of known-malicious domains and DNS-based data tunneling (cloud-native: Route 53 Resolver DNS Firewall, GCP Cloud DNS policies, Azure Firewall DNS proxy).
- **Network Firewall in front of the cluster** — AWS Network Firewall / GCP Cloud NGFW / Azure Firewall for L7 inspection of egress traffic.

7. Workload and supply-chain security

What you run is at least as important as where you run it. The supply chain — from base image to deployed pod — has become a primary attack surface.

Image hardening — ten things to get right

- **Minimal base images** — distroless, scratch, or Alpine where appropriate. Less attack surface; smaller transfer; faster startup.
- **Non-root user** — `USER 10001` (any high UID); never run as root unless you have a specific reason.
- **Read-only root filesystem** — set `readOnlyRootFilesystem: true` in pod spec; mount tmpfs for any writable paths.
- **Drop all Linux capabilities**, then add only what you need. `capabilities.drop: ["ALL"]`; `capabilities.add: ["NET_BIND_SERVICE"]` if binding to port 80.
- **seccomp RuntimeDefault** — restricts the syscalls the container can make. The default profile blocks most kernel-exploit primitives.
- **AppArmor / SELinux profile** — layered mandatory access control on top of seccomp.
- **No unnecessary tools** — don't ship curl, wget, busybox, or shells in production images. They're attacker-favorite living-off-the-land tools.
- **Pin everything by digest**, not by tag. `image@sha256:...` not `image:latest`.
- **Scan continuously** — not just at build, but at registry storage time and at admission time. CVEs are discovered every day.
- **Reproducible builds** — the same source should always produce the same image. Build provenance attestations link an image back to its source commit.

A hardened pod spec

```
apiVersion: v1
kind: Pod
metadata:
  name: api
  namespace: prod-app
spec:
  automountServiceAccountToken: false   # only if pod doesn't need K8s API access
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: api
    image: registry.yoursaas.com/api@sha256:abcd...
    securityContext:
      allowPrivilegeEscalation: false
      privileged: false
      readOnlyRootFilesystem: true
      runAsNonRoot: true
      capabilities:
        drop: ["ALL"]
    resources:
      requests: {cpu: 100m, memory: 128Mi}
      limits:   {cpu: 500m, memory: 512Mi}
    volumeMounts:
    - name: tmp
      mountPath: /tmp
  volumes:
  - name: tmp
    emptyDir: {medium: Memory}
```

The four supply-chain control points

Supply-chain security in Kubernetes spans four control points, each closing a class of attack:

Control point	Closes	Tools
Source	Compromised code via insecure repo, missing review	Branch protection; signed commits; CODEOWNERS
Build	Malicious build steps; injection into pipeline	Hermetic builds; SLSA Level 3+; ephemeral runners; provenance attestation
Registry	Image tampering; tag re-pointing; unsigned content	Private registry; image scanning at push; immutable tags; signature storage
Admission	Unsigned or vulnerable images running	Sigstore/cosign verification; Binary Authorization; Kyverno verifyImages

Image signing with Sigstore (cosign)

Sigstore has become the open-source standard for signing container images and other artifacts. The flow is: your build pipeline signs the image (either with a stored key or, better, with a short-lived keyless certificate issued by Sigstore's Fulcio based on the pipeline's OIDC identity). The signature is stored alongside the image in the registry. At admission time, the cluster's policy engine verifies the signature against your trust policy.

```
# In CI (GitHub Actions, keyless signing with OIDC):
cosign sign registry.yoursaas.com/api@sha256:abcd... \
  --identity-token=$ACTIONS_ID_TOKEN_REQUEST_TOKEN

# At admission time (Kyverno policy verifies):
verifyImages:
- imageReferences: ["registry.yoursaas.com/*"]
  attestors:
  - entries:
    - keyless:
        subject: "https://github.com/yourorg/api/.github/workflows/build.yml@refs/heads/main"
        issuer: "https://token.actions.githubusercontent.com"
```

Cloud-native supply-chain controls

Capability	EKS	GKE	AKS
Native registry with scanning	ECR with native scanning (basic / enhanced)	Artifact Registry with vulnerability scanning	ACR with native scanning (Defender for Containers)
Image signing service	ECR + cosign / AWS Signer for container images	Artifact Registry + cosign / Google Binary Authorization	ACR + cosign / Notation
Admission-time policy enforcement	Kyverno / Gatekeeper / VAP (no AWS-native equivalent)	Binary Authorization (native, integrated with GKE)	Kyverno / Gatekeeper / Image Integrity (preview)
SBOM generation & storage	Inspector + ECR enhanced scanning	Artifact Analysis + SBOM	Defender for Containers + ACR SBOM
Build provenance (SLSA)	CodeBuild attestations / GitHub Actions OIDC	Cloud Build with provenance attestations (native)	GitHub Actions OIDC to ACR

Container runtime sandboxes

Standard containers share the host kernel. For workloads that handle untrusted code, hostile multi-tenancy, or compliance requiring stronger isolation, two sandboxing approaches add a layer of kernel-level isolation:

- **gVisor (runsc)** — a user-space kernel that intercepts syscalls from the container and implements a Linux-compatible subset, dramatically reducing the host kernel attack surface. Available in GKE (Sandbox Pods), EKS via add-ons, AKS via configuration.
- **Kata Containers** — each pod runs in a lightweight VM with its own kernel. Stronger isolation than gVisor (full kernel separation) at higher overhead. Available across managed K8s as a runtime class.
- **Firecracker (used by AWS Fargate)** — microVM isolation; not directly user-configurable in EKS but underpins Fargate.

8. Secrets management

Kubernetes Secrets are base64, not encryption. Treat them as one option among several — and never the storage of record for sensitive credentials.

What native Kubernetes Secrets are (and aren't)

A Kubernetes `Secret` is just a key-value object stored in etcd. By default it is **base64 encoded, not encrypted**. When etcd encryption at rest is enabled (which it is by default in all managed K8s services), Secrets are encrypted at the storage layer — but they are decrypted when read by the API server and presented in cleartext to any pod or user with read access. That makes RBAC on Secrets the primary defense, and external secret stores the right answer for high-value credentials.

Three approaches, in increasing strength

Approach	Strength	When to use
Native K8s Secret + etcd encryption (managed key)	Acceptable for low-sensitivity values (TLS certs for internal services, non-prod config)	Baseline; the default that ships with every cluster
Native K8s Secret + envelope encryption with customer-managed KMS	Acceptable for moderate sensitivity; CMK separates trust from cloud provider operators	Production clusters with regulatory pressure
External secret store (Secrets Manager / Secret Manager / Key Vault) + CSI driver	Strong — secrets never enter etcd; rotation is centralized; audit trail per access	Production; any secret you'd rotate quarterly or that has compliance scope

The Secrets Store CSI Driver pattern

The Secrets Store CSI Driver mounts secrets from an external store as files (or syncs them into a K8s Secret) at pod startup. The secret never lives in etcd, the pod uses its workload identity (IRSA / Workload Identity / Azure Workload Identity) to authenticate to the store, and rotation is centralized in the store rather than in cluster manifests.

```
# Example: AWS Secrets Manager via CSI driver
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: db-creds
  namespace: prod-app
spec:
  provider: aws
  parameters:
    objects: |
      - objectName: prod/db/password
        objectType: secretsmanager
---
apiVersion: v1
kind: Pod
metadata: {name: api, namespace: prod-app}
spec:
  serviceAccountName: api-sa   # bound to IAM role via IRSA
  containers:
  - name: api
    image: registry.yoursaas.com/api@sha256:abcd...
    volumeMounts:
    - name: secrets
      mountPath: /etc/secrets
      readOnly: true
  volumes:
  - name: secrets
    csi:
      driver: secrets-store.csi.k8s.io
      readOnly: true
      volumeAttributes:
        secretProviderClass: db-creds
```

Cloud-native secret stores

Capability	EKS	GKE	AKS
Native secret store	AWS Secrets Manager / Parameter Store	Google Secret Manager	Azure Key Vault
CSI driver provider	AWS provider for Secrets Store CSI Driver	GCP provider / Secret Manager add-on	Azure Key Vault provider for Secrets Store CSI Driver
Workload identity for auth	IRSA / Pod Identity	GKE Workload Identity	Azure Workload Identity
Auto-rotation in the store	Native (Lambda-backed rotation)	Native (replication; rotation via Cloud Functions)	Native (via Key Vault rotation policies)
Auto-sync to mounted pod	Via reloader / sync-as-K8s-Secret + restart	Same pattern	Same pattern

Anti-pattern: secrets in ConfigMaps or environment variables

Never put credentials in ConfigMaps — they are unencrypted and treated as non-sensitive by tooling. **Avoid** environment variables for secrets — they leak via process listings, crash dumps, and child processes that inherit the environment. Mount secrets as files (read-only) and read them at startup.

9. Runtime security and detection

Even with admission control, hardened images, and tight RBAC, attackers succeed. Runtime detection is what catches them in the act — and increasingly stops them.

What “runtime security” actually means

Runtime security observes what containers actually do once they're running — the syscalls they make, files they open, network connections they establish, processes they spawn — and compares that activity against rules or learned baselines. When something anomalous happens (a shell spawned in a production container, a sensitive file read, an unexpected outbound connection), the system alerts or blocks. This is the layer that catches what admission control missed.

The eBPF revolution

Modern runtime security tools are built on **eBPF** (extended Berkeley Packet Filter), which lets safe programs run in the Linux kernel and observe syscalls, network events, and file activity with near-zero overhead. eBPF replaced the older kernel-module approach (which required elevated privileges and risked kernel panics) and underpins Cilium, Falco's modern driver, Tetragon, Pixie, and most cloud-native runtime security tools.

Open-source runtime security tools

Tool	Approach	Strengths
Falco	eBPF/kernel-module syscall monitoring + rules engine (YAML)	CNCF graduate; mature rule library; broad detection coverage
Tetragon (Isovalent / Cilium)	eBPF-native; can enforce (kill processes), not just detect	Performance; ties to Cilium network identity; in-kernel enforcement
Sysdig (commercial, OSS-derived)	Falco-based + commercial features (forensic capture, compliance)	Enterprise support; deep forensics
KubeArmor	AppArmor / SELinux / eBPF for in-line enforcement	Policy-driven prevention, not just detection
Tracee (Aqua)	eBPF; signature library for known attack patterns	Strong threat-research-driven signature set

Falco rule example — detect shell-in-container

```
- rule: Terminal shell in container
  desc: A shell was spawned inside a container by an unexpected process
  condition: >
    spawned_process and container
    and shell_procs
    and proc.tty != 0
    and container_entrypoint
    and not user_expected_terminal_shell_in_container_conditions
  output: >
    A shell was spawned in a container (user=%user.name container_id=%container.id
    container_name=%container.name shell=%proc.name parent=%proc.pname
    cmdline=%proc.cmdline pid=%proc.pid)
  priority: NOTICE
  tags: [container, shell, mitre_execution]
```

Cloud-native runtime defense

Capability	EKS	GKE	AKS
Native runtime threat detection	Amazon GuardDuty EKS Runtime Monitoring	Google Cloud Security Command Center — Container Threat Detection	Microsoft Defender for Containers
Mechanism	eBPF agent (managed) sending findings to GuardDuty	eBPF agent + CSCC; sent to Security Command Center	Defender sensor (eBPF) + agentless scanning
Detects	Shell-in-container, crypto-mining, container escape, malicious binaries	Same plus suspicious binary execution, library load	Same plus secrets discovery, vulnerability assessment
Integration with native posture mgmt	Security Hub aggregation	Security Command Center Premium	Microsoft Defender Cloud Posture (CSPM)
Cost model	Per protected vCPU/hour	Per node-hour (Premium tier)	Per protected node/hour or vCPU

Incident response runbook — compromised pod

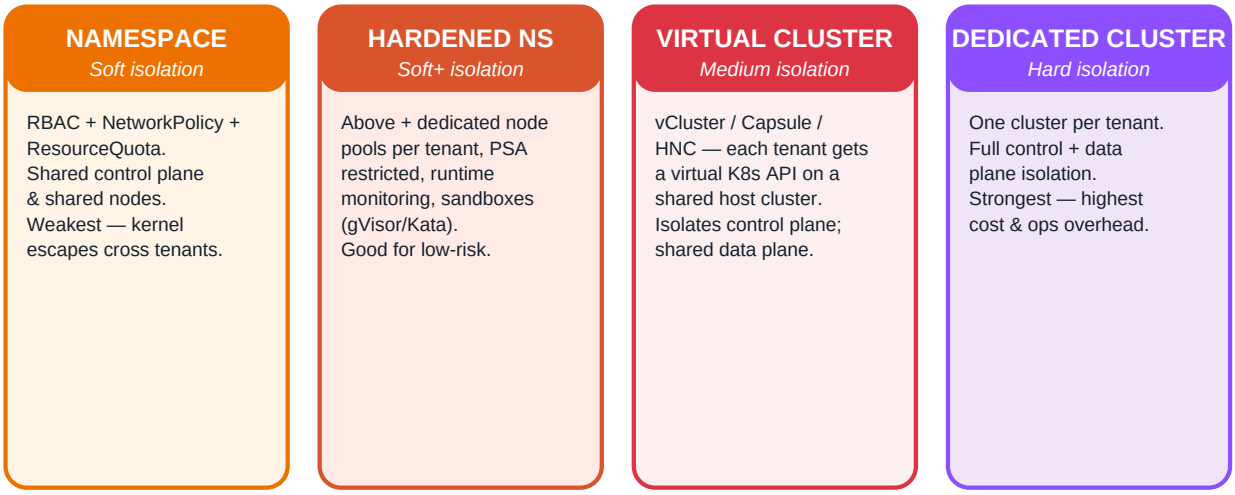
When runtime detection fires, the response should be rehearsed, fast, and (where safe) automated. A standard runbook for “suspected compromised pod”:

- **1. Detect** — Falco / GuardDuty / Defender / Tetragon fires; event enriched with pod, namespace, node, image, command.
- **2. Triage** — automated severity scoring; high-severity events page on-call, lower-severity ticket.
- **3. Isolate** — apply a NetworkPolicy that allows no traffic to/from the pod (or label the node tainted-isolated). Done in seconds; doesn't kill the pod, so forensics remain possible.
- **4. Capture** — snapshot the pod's filesystem (ephemeral container with kubectl debug), capture process list, network connections, kernel events for last N minutes.
- **5. Rotate credentials** — revoke the pod's ServiceAccount token, rotate any secrets the pod could have read.
- **6. Terminate & redeploy** — delete the pod (Deployment recreates from clean image); cordoning the node if compromise is suspected at host level.
- **7. Investigate** — review admission logs (what changed?), audit logs (who deployed?), CI/CD logs (what got built?).
- **8. Hunt** — the same indicators across the cluster; the same attacker may have other footholds.
- **9. Postmortem** — root cause, blast radius, and a control that would have prevented or shortened the incident.

10. Multi-tenancy patterns

“Can we put multiple tenants in one cluster?” Almost always yes — the real question is how strong the isolation must be, and what you'll pay for it.

Multi-tenancy isolation — from weakest to strongest



Choosing an isolation level

Multi-tenancy strength is a spectrum, and different tenants in the same product can sit at different points. The diagram above shows the canonical four levels. The right choice depends on the threat model: are your tenants mutually trusting (a SaaS where tenants are paying customers), partially trusting (a platform-as-a-service for internal teams), or actively adversarial (you run untrusted code on behalf of users)? Adversarial multi-tenancy demands far stronger isolation than the SaaS case.

Level	Cost & complexity	Adversary it withstands
Namespace + RBAC + NetworkPolicy	Lowest — one cluster, shared everything	Configuration mistakes; same-cluster lateral movement attempts. Not kernel exploits.
Hardened namespace + dedicated nodes + PSA restricted + sandbox	Moderate — more nodes; sandbox overhead	Most cross-tenant compromises in benign-mistake or low-skill threat models
Virtual cluster (vCluster, Capsule, HNC)	Moderate-high — extra control planes per tenant	Tenant-side cluster-API mistakes; partial control-plane isolation
Dedicated cluster per tenant	Highest — cost, ops overhead per tenant	Adversarial multi-tenancy; regulatory mandates; the strongest stance

Controls that strengthen namespace-level isolation

- **One ServiceAccount per workload**, never default. Tenant workloads should never assume each other's ServiceAccount tokens.
- **NetworkPolicy default-deny across the namespace**, with explicit allow-lists — prevents tenant A from probing tenant B's pods.
- **Dedicated node groups per tenant (or per tenant tier)** using taints + tolerations + nodeSelectors — ensures workloads don't share kernels.
- **ResourceQuota and LimitRange per namespace** — stops one tenant from starving the cluster (noisy-neighbor / DoS).
- **PSA restricted** for all tenant namespaces — removes the privileged-pod escape path.
- **Runtime security agent per node** — detects cross-tenant escape attempts in flight.
- **Per-tenant CMK for any secrets / volumes** — even if RBAC is bypassed, data remains encrypted under a key only the right tenant can use.

11. Logging, audit, and incident response

If you can't reconstruct what happened, you don't know whether the cluster is secure. Logging is part of the security design, not a separate concern.

Five log streams every cluster must capture

Log stream	Source	What it tells you
Kubernetes API audit	API server	Every API call: who, what, when, response. Forensic record of cluster activity.
Container runtime / kubelet	Each node	Pod lifecycle events; container starts, OOMs, crashes. Operational + security.
Application logs (stdout/stderr)	Pods, via node-level forwarder	App behavior and errors. Usually shipped to SIEM.
Network flow logs	VPC / CNI flow records, Hubble (Cilium)	Pod-to-pod and egress flows. Useful for detection and forensics.
Runtime security events	Falco / GuardDuty / Defender / Tetragon	Syscall- and behavior-level events. Detection signal.

Audit log configuration — the right level of detail

Kubernetes audit policies define which events to record at which level (None, Metadata, Request, RequestResponse). “Log everything at RequestResponse” is impossibly expensive; “log Metadata only” misses critical detail. The right policy logs different levels for different events:

```
# Excerpt: production audit policy
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
# Don't log routine read activity from system components
- level: None
  users: ["system:kube-proxy", "system:kube-scheduler"]
  verbs: ["watch", "get", "list"]

# Log secret access with full Request detail — critical for forensics
- level: Request
  resources:
  - group: ""
    resources: ["secrets"]

# Log RBAC changes with full RequestResponse — must be reconstructible
- level: RequestResponse
  resources:
  - group: "rbac.authorization.k8s.io"

# Log exec/portforward at Request level — catches privilege abuse
- level: Request
  resources:
  - group: ""
    resources: ["pods/exec", "pods/portforward", "pods/attach"]

# Default: log everything at Metadata level
- level: Metadata
```

Log routing & retention

Logs must reach a tamper-resistant destination, separate from cluster operators' day-to-day reach. The standard pattern is to ship logs to a dedicated, locked-down account / project / subscription, write them to object storage with immutability (Object Lock, retention policies), and feed them into a SIEM for analysis. Retention should match the longest investigation window you'd plausibly need — typically one year minimum for audit logs, three to seven for regulated industries.

Capability	EKS	GKE	AKS
Native log destination	CloudWatch Logs	Cloud Logging	Azure Monitor / Log Analytics
Long-term archive	S3 with Object Lock (WORM)	GCS with retention policy / Bucket Lock	Azure Storage with immutable policy
SIEM integration	Security Hub / OpenSearch / 3rd party	Chronicle (Google SecOps) / 3rd party	Microsoft Sentinel (native) / 3rd party
Cross-account isolation	Logs in a dedicated log-archive account	Logs in a dedicated security project	Logs in a dedicated security subscription

12. The EKS / GKE / AKS comparison

The full security-feature matrix across the three major managed Kubernetes services. Use this to translate any control in this reference to your platform.

Cluster topology & control plane

Aspect	EKS	GKE	AKS
Auto/managed-node mode	EKS Auto Mode (newer); managed node groups; Fargate	GKE Autopilot (pod-only); Standard with node-auto-provisioning	AKS Automatic (preview); auto-upgrade channels
Default node OS	Bottlerocket / Amazon Linux 2023	Container-Optimized OS (COS)	Azure Linux (Mariner) / Ubuntu
Default CNI	AWS VPC CNI	GKE Dataplane V2 (Cilium-based)	Azure CNI / Azure CNI Powered by Cilium
Private control-plane endpoint	Yes — EndpointPublicAccess=false	Yes — private cluster + master authorized networks	Yes — private cluster + Private Link
etcd customer-managed key encryption	KMS envelope encryption for Secrets	Cloud KMS application-layer secrets encryption	Azure Key Vault KMS plugin
Release / channel model	Standard support + extended support	Rapid / Regular / Stable release channels	Stable / Rapid / Long-term-support channels

Identity & access

Aspect	EKS	GKE	AKS
Cluster user authn	IAM (via aws-auth or Access Entries)	Google IAM (Cloud Identity) or IAM Workforce Pools	Microsoft Entra ID (native)
Workload identity	Pod Identity (newer) / IRSA	Workload Identity	Azure Workload Identity
RBAC integration with cloud groups	IAM role/group → K8s group mapping	Google Groups for RBAC	Entra ID groups — native
Just-in-time elevation	IAM Identity Center + session policies	IAM conditional access; Boundary/Teleport	Entra Privileged Identity Management (PIM)

Workload, supply chain & runtime

Aspect	EKS	GKE	AKS
Native registry	ECR	Artifact Registry	ACR
Native image scanning	ECR scanning (basic + enhanced via Inspector)	Artifact Analysis	Defender for Containers
Admission-time signature verification	Kyverno / Gatekeeper / VAP (no AWS-native)	Binary Authorization (native)	Kyverno / Gatekeeper / Image Integrity

Aspect	EKS	GKE	AKS
Sandbox runtimes	Add-on (gVisor, Kata)	GKE Sandbox (gVisor) native	Kata via runtime class
Native runtime threat detection	GuardDuty EKS Runtime Monitoring	Container Threat Detection (SCC Premium)	Defender for Containers

Network & policy

Aspect	EKS	GKE	AKS
NetworkPolicy enforcement	Add-on (Cilium / Calico) or native VPC CNI policy	Native (Dataplane V2 / Cilium)	Azure NPM or Cilium
L7 / FQDN policy	Cilium add-on	Native (Dataplane V2)	Azure CNI Powered by Cilium
Service mesh native option	App Mesh (deprecated path); BYO Istio / Linkerd	Cloud Service Mesh (managed Istio)	Open Service Mesh (deprecated); BYO Istio
Network firewall / egress inspection	AWS Network Firewall + Cloud NAT	Cloud NGFW + Cloud NAT	Azure Firewall + AKS egress-type=userDefinedRouting

13. Design principles and decision guide

The principles that hold across every cluster, regardless of cloud, and the decisions that matter most to get right early.

The ten principles

- **Default-deny everywhere.** Network policy, RBAC, admission, egress. The network is flat, the default ServiceAccount is over-privileged, and pods can call anything — unless you say otherwise. Say otherwise, deliberately.
- **Treat the API server as the most sensitive system you own.** Private endpoint, authorized networks, audit logging at full fidelity, MFA-protected human access, no static kubeconfigs.
- **Identity, not credentials.** Workload identity for pods (IRSA / Workload Identity / Azure WI). SSO + short-lived tokens for humans. No long-lived access keys anywhere.
- **Layer admission control.** PSA for the basics; OPA / Kyverno / VAP for policy; image signature verification for supply chain. Each layer catches what the others miss.
- **Harden every pod by default.** Non-root, read-only root FS, drop all capabilities, seccomp RuntimeDefault, resource limits, no host namespaces. Make this the cluster-wide default, not a per-workload exercise.
- **Trust no image you can't verify.** Private registry, signed images, admission-time verification, SBOM stored and queryable, continuous CVE scanning.
- **Secrets live outside the cluster.** External secret store (Secrets Manager / Secret Manager / Key Vault) accessed via CSI driver with workload identity. Native K8s Secrets are for non-sensitive values only.
- **Observe everything. Detect what matters.** Audit logs at full fidelity, runtime security agent on every node, anomalies routed to SIEM, automated containment for known-bad behavior.
- **Multi-tenancy is a spectrum.** Match the isolation level to the threat model. Don't pay for dedicated clusters when hardened namespaces suffice; don't pretend namespaces are enough for adversarial workloads.
- **Practice the incident.** Runbooks for compromised pods, leaked credentials, exposed API servers. Quarterly game-days. Mean-time-to-contain is a real metric.

Decision guide — the questions worth fighting for early

Question	The right default	When to choose differently
Public API endpoint?	No — private only, or restricted allow-list	Never — there is no good reason in production
Default ServiceAccount mounted?	No — automountServiceAccountToken: false unless needed	When the pod genuinely needs K8s API access
Cluster-admin bindings?	Tiny number of break-glass identities only	Never expand without a quarterly review
PSA enforcement level?	Restricted on all app namespaces	Baseline only for genuinely legacy workloads being migrated
External or native secret store?	External (Secrets Manager / Secret Manager / Key Vault) via CSI	Native is acceptable for low-sensitivity, ephemeral values

Question	The right default	When to choose differently
Service mesh?	No — start with NetworkPolicy + workload identity	When mTLS-everywhere is a hard requirement or L7 authz needed
Image signing?	Yes — cosign + admission verification	There is no good reason to skip this in 2026
Multi-tenancy model?	Hardened namespaces + dedicated node groups + PSA restricted	Dedicated cluster for adversarial workloads or regulatory mandate
Runtime security?	Yes — cloud-native (GuardDuty/SCC/Defender) or Falco	There is no good reason to skip this in 2026

The one-paragraph summary

Production Kubernetes security is a layered program, not a feature. Lock down the control plane (private endpoint, audit at fidelity, no static kubeconfigs). Use workload identity for pods and SSO for humans — never long-lived credentials. Enforce admission policy in depth (PSA restricted, OPA/Kyverno/VAP for org policy, signature verification for images). Default-deny the network and the secrets. Run a runtime security agent on every node. Ship audit logs to immutable storage with a year of retention. Practice incident response. The cloud-specific details (EKS Pod Identity vs GKE Workload Identity vs AKS Workload Identity; GuardDuty vs SCC vs Defender) matter operationally, but the principles are the same: zero trust, least privilege, defense in depth, demonstrable audit. Get those right, and the cluster is defensible. Skip them, and no amount of vendor-specific wizardry will save you.